🏛️ SheetDB

A Schema-Driven Relational Database Framework for Google Apps Script

SheetDB is a multi-layered framework designed to transform Google Sheets into a structured, relational database environment. It provides automated schema provisioning, high-performance batch operations, and a dynamic ORM for complex data relationship management.

Table of Contents

Installation

Back to Table of Contents

Deploy the `SheetDB` framework using one of three primary methods depending on your development environment and dependency requirements.

Option A: Apps Script Library

Connect to the hosted library version to receive automatic updates and minimize project file clutter.

Prerequisites:

- A Google Apps Script project.
- The `SheetDB` Script ID.

Steps:

1. Open the Google Apps Script editor.
2. Click the **+** icon next to **Libraries**.
3. Enter the **Script ID**: `YOUR_SCRIPT_ID_HERE`.
4. Click **Look up** and select the latest stable version.
5. Set the identifier to `SheetDB` and click **Add**.

Option B: Manual File Inclusion

Copy the consolidated framework source code directly into your project for a zero-dependency setup.

Prerequisites:

- Access to the `SheetDB.js` distribution file.

Steps:

1. Download or copy the content of `SheetDB.js` from the repository root.
2. Create a new script file in your target project named `SheetDB_Library.gs`.
3. Paste the entire content of `SheetDB.js` into the file and save.

Option C: Clasp/Git Deployment

Clone the source repository and deploy using the `clasp` CLI for professional version-controlled workflows.

Prerequisites:

- clasp installed and authenticated.
- Git installed on your local machine.

Usage Example:

```
git clone https://github.com/your-org/SheetDB.git
cd SheetDB
```

```
clasp push
```

Expected Output:

```
# Pushing files to Google Drive...
# └─ SheetDB/index.js
# └─ ...
# Pushed 12 files.
```

Quick Start

Back to Table of Contents

Establish a functional database environment by defining a schema and executing the automated provisioning engine.

1. Define Database Schema

Create a configuration object that describes your table structures, primary keys, and column constraints.

Usage Example:

```
const mySchema = {
  "version": "1.0.0",
  "database": "DazzlingDB",
  "categories": {
    "General": {
      "tables": {
        "Student": {
          "primaryKey": "student_id",
          "columns": {
            "student_id": { "type": "string", "required": true },
            "name": { "type": "string", "required": true },
            "email": { "type": "string", "format": "email" }
          }
        }
      }
    }
  }
};
```

2. Initialize and Provision

Instantiate the `SheetDB` engine and call the `provision` method to generate the physical spreadsheets and headers.

Syntax/Parameters Table:

Parameter	Type	Description
`folderId`	`string`	The Google Drive Folder ID where spreadsheets will be stored.
`schema`	`Object`	The JSON schema configuration object.

Usage Example:

```
function setup() {
  const FOLDER_ID = "1abc123...your_id";

  // Initialize the database facade
  const db = SheetDB.init(FOLDER_ID, mySchema);

  // Build missing spreadsheets and headers
  db.setup.provision();

  Logger.log("Database initialized successfully.");
}
```

Expected Output:

```
[SheetDB] Creating Category: General
[SheetDB] Table 'Student' created with 3 columns.
Database initialized successfully.
```

3. Execute First Insert

Use the dynamically generated repository to persist a data object into the spreadsheet.

Usage Example:

```
function addStudent() {
  const db = SheetDB.init(FOLDER_ID, mySchema);
```

```
  // Insert data via the 'Student' repository
  const student = db.Student.insert({
    student_id: "STU-001",
    name: "Moni",
    email: "moni@example.com"
  });

  Logger.log(JSON.stringify(student, null, 2));
}
```

Expected Output:

```
{
  "student_id": "STU-001",
  "name": "Moni",
  "email": "moni@example.com",
  "__rowNumber": 2
}
```

Unified API & Queries

Back to Table of Contents

The Unified API provides a standard application service layer using Generic Actions to interact with any entity in the database via a uniform interface.

The Response Envelope

All API actions return a standardized JSON structure to ensure predictable integration with frontend applications.

Syntax/Parameters Table:

Property	Type	Description
`success`	`boolean`	Indicates if the operation was executed without errors.
`action`	`string`	The name of the action performed (e.g., `query`, `create`).

Property	Type	Description
`data`	`Object|Array`	The resulting payload or `null`.
`error`	`string`	Contains the error message if `success` is false.

Generic Query Action

Execute collection-level searches using declarative JSON filters to retrieve specific subsets of data.

Syntax/Parameters Table:

Parameter	Type	Description
`entity`	`string`	**Required.** The name of the table to query (e.g., `Student`).
`filter`	`string`	**Optional.** A JSON-stringified object defining equality filters.

Usage Example:

```
function fetchActiveStudents() {
  const db = SheetDB.init(FOLDER_ID, mySchema);

  const action = new GenericQueryAction({
    db: db,
    params: {
      entity: "Student",
      filter: JSON.stringify({ status: "active" })
    }
  });

  const response = action.run();
  Logger.log(JSON.stringify(response, null, 2));
}
```

Expected Output:

```
{
  "success": true,
  "action": "query",
  "data": [
    { "student_id": "S1", "name": "Moni", "status": "active" }
  ]
}
```

Generic Retrieve Action

Perform high-performance single-record lookups using the entity's primary key.

Syntax/Parameters Table:

Parameter	Type	Description
`entity`	`string`	**Required.** The target table name.
`id`	`any`	**Required.** The primary key value of the record.

Usage Example:

```
function getStudent() {
  const db = SheetDB.init(FOLDER_ID, mySchema);

  const action = new GenericRetrieveAction({
    db: db,
    params: { entity: "Student", id: "STU-001" }
  });

  const response = action.run();
  Logger.log(JSON.stringify(response, null, 2));
}
```

Expected Output:

```
{
  "success": true,
  "action": "retrieve",
  "data": { "student_id": "STU-001", "name": "Moni" }
}
```

Generic Create/Update/Delete Actions

Apply state changes to the database using universal handlers for insertion, modification, and removal.

Usage Example (Update):

```
function updateStudent() {
  const db = SheetDB.init(FOLDER_ID, mySchema);

  const action = new GenericUpdateAction({
    db: db,
    params: {
      entity: "Student",
      id: "STU-001",
      data: JSON.stringify({ name: "Moni Refactored" })
    }
  });

  const response = action.run();
  Logger.log(response.success ? "Success" : response.error);
}
```

Expected Output:

```
{ "success": true, "action": "update", "data": { "student_id": "STU-001", "name":
"Moni Refactored" } }
```

Advanced Repository Patterns

Back to Table of Contents

The repository layer offers sophisticated patterns for handling complex data structures and high-volume operations. These methods move beyond simple row mapping to provide "Document-to-Relational" orchestration, allowing you to interact with your database using natural, hierarchical JSON objects.

Nested Relational Inserts (insertOne)

The `insertOne` method allows for the simultaneous persistence of a primary entity and all its associated relational children in a single command. The engine automatically parses the input payload, identifies nested objects or arrays that match defined relationships, and executes a multi-table save sequence.

A critical feature of this method is **Automatic Foreign Key Injection**. Before saving nested children, the engine captures the Primary Key of the newly saved parent and injects it into the appropriate fields of the child objects. This eliminates the manual effort of managing IDs across related sheets.

Usage Example:

```
function nestedInsertDemo() {
  const db = SheetDB.init(FOLDER_ID, mySchema);

  // Save Student, Address, and 2 Enrollments in one call
  const payload = {
    student_id: "STU-NESTED-001",
    name: "Advanced User",
    address: {
      address_id: "ADDR-001",
      city: "San Francisco",
      state: "CA"
    },
    enrollment: [
      { enrollment_id: "ENR-001", item_id: "COURSE-A" },
      { enrollment_id: "ENR-002", item_id: "COURSE-B" }
    ]
  };

  const student = db.Student.insertOne(payload);
  Logger.log("Graph saved for: " + student.name);
}
```

Expected Output:

```
[SheetDB] Parent 'Student' saved. PK 'STU-NESTED-001' captured.
[SheetDB] Injecting FK into relation 'address'.
[SheetDB] Injecting FK into relation 'enrollment' (2 items).
[SheetDB] Nested write complete.
```

High-Performance Bulk Import (insertMany)

Designed for massive data migrations and large-scale imports, the `insertMany` method utilizes optimized batch-writing techniques to minimize Google Sheets API overhead. Instead of making individual network requests for every record, this method groups data into a single, high-speed 2D array operation.

By leveraging the `BatchBucket` internal engine, `insertMany` provides significant performance gains, reducing execution time from minutes to seconds for datasets exceeding several hundred rows. It supports both flat arrays and complex nested documents, maintaining full relational integrity even during bulk operations.

Usage Example:

```
function bulkImport() {
  const db = SheetDB.init(FOLDER_ID, mySchema);
  const students = [
    { student_id: "B-1", name: "User 1", status: "active" },
    { student_id: "B-2", name: "User 2", status: "active" },
    // ... up to 1000+ records
  ];

  const results = db.Student.insertMany(students);
  Logger.log(`Successfully imported ${results.length} students.`);
}
```

Priority Transaction Orchestration (BatchBucket)

The `BatchBucket` serves as a "Unit of Work" buffer that orchestrates how data is physically committed to Google Drive. It is responsible for managing the **Insertion Sequence**, ensuring that dependencies are respected by writing parent tables (like `Student`) before child tables (like `Enrollment`).

This prioritized execution model prevents "Orphaned Records" by ensuring that referential links are established in the correct physical order. The `BatchBucket` also acts as the primary gatekeeper for the system's atomic validation and transaction rollback strategies, which protect the database during partial failures.

Sequence Logic Table:

Order	Level	Responsibility
1	**Parent**	High-level entities that provide Primary Keys (e.g., `Student`, `Course`).
2	**Child**	Entities that depend on Parent IDs (e.g., `Address`, `Enrollment`, `Payment`).
3	**Commit**	Finalization of transaction markers and status updates.

The Relational Graph & ORM

Back to Table of Contents

The Relational Graph layer transforms flat spreadsheet rows into a connected web of "Smart Objects." By utilizing the metadata defined in your schema, the ORM dynamically injects navigation logic into your models, allowing you to traverse complex many-to-one and one-to-many relationships without writing manual lookup queries.

Relational Configuration

Relationships are established by defining a `relations` block within a table's schema. Each entry specifies a logical name, the type of connection, the target entity, and the Foreign Key column used to bridge the two tables. This declarative approach allows the engine to understand the directional flow of data across different spreadsheets.

Supported Relation Types:

Type	Cardinality	Direction	Logic
`belongsTo`	N:1	Child → Parent	The source record contains the FK. Finds one parent.
`hasMany`	1:N	Parent → Child	The target table contains the FK. Finds many children.
`hasOne`	1:1	Parent → Child	The target table contains the FK. Finds one unique child.

Usage Example (Schema):

```
"Student": {
  "primaryKey": "student_id",
  "columns": { ... },
  "relations": {
    "address": { "type": "hasOne", "target": "Address", "foreignKey": "student_id"
},
    "enrollments": { "type": "hasMany", "target": "Enrollment", "foreignKey":
"student_id" }
  }
}
```

Walking the Graph (Auto-Injected Methods)

Upon hydration, every `BaseModel` instance is automatically "upgraded" with methods corresponding to its defined relations. These methods act as fluent gateways, allowing you to fetch related records directly from the model object. This pattern significantly reduces boilerplate code and makes the application logic more intuitive and readable.

Usage Example:

```
function walkTheGraph() {
  const db = SheetDB.init(FOLDER_ID, mySchema);
  const student = db.Student.findById("STU-001");

  // Fetch the student's unique address (1:1)
  const addr = student.address();

  // Fetch the student's collection of enrollments (1:N)
  const list = student.enrollments();

  Logger.log(`Student in ${addr.city} has ${list.length} enrollments.`);
}
```

Expected Output:

```
Student in San Francisco has 2 enrollments.
```

Manual Resolution (The Database API)

For scenarios requiring explicit control or high-level orchestration, the main database facade provides a direct `resolve` method. This utility is particularly useful for debugging or building generic services that need to traverse relationships without relying on the auto-injected instance methods.

Syntax/Parameters Table:

Parameter	Type	Description
`model`	`BaseModel`	The source object to start the traversal from.
`relationName`	`string`	The key of the relation defined in the schema.

Usage Example:

```
function explicitResolve() {
  const enrollment = db.Enrollment.findById("ENR-001");

  // Climb back up to the parent student manually
  const parent = db.resolve(enrollment, 'student');

  Logger.log("Resolved parent: " + parent.name);
}
```

Under the Hood: Lazy Loading

To ensure optimal performance and prevent excessive Google Apps Script execution time, SheetDB utilizes a **Lazy Loading** strategy. Related records are not physically fetched from the spreadsheets until the specific relation method is invoked.

This approach ensures that loading a "Student" record does not trigger a cascade of reads for their address, payments, and attendance unless your code explicitly asks for them. This makes the ORM safe for memory-constrained environments like Google Sheets while maintaining a powerful relational interface.

Safety, Validation & Transactions

Back to Table of Contents

The Safety Engine provides industrial-grade protection for your data integrity. By combining declarative schema rules with an identity-based transaction model, SheetDB ensures that only valid data enters the system and that partial failures never leave your spreadsheets in a corrupted state.

Declarative Validation Rules

Field-level constraints are defined directly within your JSON schema, allowing the engine to automatically enforce data types, character limits, and formatting requirements. These rules are powered by the standalone `Validate` utility, ensuring consistent logic across both the server-side database and client-side form applications.

Supported Validation Keys:

Rule	Type	Description
`required`	`boolean`	Ensures the field is present and non-empty.
`minLength` / `maxLength`	`number`	Enforces character count boundaries on strings.
`min` / `max`	`number`	Enforces value ranges on numeric fields.
`format: "email"`	`string`	Validates value against standard email regex patterns.
`choices`	`Array`	Restricts values to a strict list of allowed options (Enum).

Exhaustive Audit Reports

SheetDB utilizes an "Audit-First" validation strategy. When a batch operation is initiated, the engine performs a comprehensive sweep of all records, collecting every violation into a single structured report rather than failing on the first error. This "Atomic Failure" model guarantees that zero physical writes occur unless the entire payload is 100% compliant.

Usage Example:

```
function validationDemo() {
  const db = SheetDB.init(FOLDER_ID, mySchema);

  // Attempt to insert an invalid record
  try {
    db.Student.insert({ student_id: "", email: "not-an-email" });
  } catch (e) {
    // e contains the BatchValidationError report
    Logger.log(e.message);
  }
}
```

Expected Output:

```
[
  {
    "table": "Student",
    "index": 0,
    "field": "student_id",
    "message": "Field is required but missing or empty."
  },
  {
    "table": "Student",
    "index": 0,
    "field": "email",
    "message": "Invalid email format."
  }
]
```

Transaction Integrity & Rollback

To prevent orphaned records during multi-table operations, SheetDB implements an **Identity-Based Transaction** model. Every record in a batch is tagged with a unique `__tx_id` and a `__tx_status` (PENDING). This metadata allows the engine to track exactly which rows belong to a specific transaction across different spreadsheets.

If a failure occurs during a multi-step write (e.g., a network timeout or quota limit), the engine triggers a **Deterministic Rollback**. Using the unique transaction ID, it surgically removes only the rows associated with the failed session, restoring your database to its exact previous state without affecting concurrent user data.

System Columns (Auto-Injected):

Column	Type	Description
`__tx_id`	`string`	The UUID that links all rows in a single logical transaction.
`__tx_status`	`enum`	Tracks state: `PENDING`, `COMMITTED`, or `FAILED`.
`__created_at`	`datetime`	The system-level timestamp for row creation.